

ЗАДАЧА ДЕТЕКЦИИ ДОРОЖНЫХ ЗНАКОВ

И. Ю. Сазонтов

Московский авиационный институт (национальный исследовательский университет)

Аннотация: В данной статье рассматривается задача детекции дорожных знаков. Работа описывает этапы детекции, то есть выделение на изображении области, в которой находится дорожный знак. Предлагается ее решение на основе метода обратного распространения ошибок с использованием датасета, который разбивается на две выборки для проведения обучения с подкреплением и тестирования. Модель проверена на переобучение.

Ключевые слова: SSD модель, датасет, обучающая выборка, тестовая выборка, классы, функция потерь, метрика MAP, переобучение, гистограмма распределения

Введение

В городских условиях количество дорожных знаков велико, они предоставляют ценную информацию для навигации. Однако из-за проблем с разрезанными элементами формы, размытой текстурой и большими углами наклона дорожные знаки не могут быть полностью восстановлены с помощью наклонной фотограмметрии, даже с использованием самых современных алгоритмов. Плохое распознавание дорожных знаков обычно приводит к менее информативному ориентированию.

В статье описаны метод нейросетевого обучения на основе модели Single-Shot Detector (SSD) [1] и его реализация, способные решить описанную проблему. Ранее для решения применялась нейросетевая модель YOLOv7 [2]. В качестве входных данных используется база данных с фотографиями российских дорог. Итогом работы является обученная модель, способная найти один из двенадцати дорожных знаков на изображении.

1. Теоретическая база

1.1. Модель SSD

Для обнаружения объектов в задачах детекции широко используется модель SSD. Она работает следующим образом: каждое входное изображение разбивается на сетки разного размера и на каждой сетке детекция выполняется для разных классов и разных соотношений сторон, что означает, что для разных типов знаков будут разные по размеру сетки, то есть если мы ищем хотя бы один знак из набора 12 разных знаков, то изображение будет поделено 12-ю разными сетками. Каждой из этих сеток присваивается оценка, которая показывает, насколько хорошо объект соответствует этой конкретной сетке, при этом применяется немаксимальное подавление (non-maximum suppression), чтобы получить окончательное обнаружение из набора перекрывающихся обнаружений.

Данная модель состоит из двух частей:

1) Базовая модель (backbone model), представляющая собой типичную предварительно обученную сеть классификации изображений, которая работает как экстрактор карты объектов. Здесь слои окончательной классификации изображений

модели удаляются, чтобы иметь в итоге только извлеченные карты объектов.

2) Верхняя часть SSD (SSD head). Здесь слои окончательной классификации изображений модели удаляются, чтобы иметь в итоге только извлеченные карты объектов.

SSD head состоит из пары сверточных слоев, сложенных вместе, и добавляется к верхней части модели дороги. Это дает вывод в виде ограничивающих рамок над объектами. Эти сверточные слои обнаруживают различные объекты на изображении. SSD основан на использовании сверточных сетей, которые создают несколько ограничивающих рамок различных фиксированных размеров и оценивают наличие экземпляра класса объекта в этих рамках, за которым следует шаг не максимального подавления для получения окончательных обнаружений.

1.2. Используемые данные

Модель была обучена на данных Russian Traffic Sign Dataset (RTSD), которые предназначены для обучения и тестирования алгоритмов распознавания дорожных знаков. Данные содержат 59188 фотографий российских дорог. Размеры фотографий варьируются от 1280x720 до 1920x1080 пикселей, они сняты в разное время года (весна, осень, зима, лето), время суток (утро, день, вечер) и при разных погодных условиях (яркое солнце, дождь, снег). Этот набор данных превосходит другие наборы данных общедоступных дорожных знаков по объему, классам знаков и изображениями знаков. Также к этому набору фотографий прилагается файл full-gt.csv, содержащий информацию о дорожных знаках на каждой фотографии [3].

Чтобы решить задачу детекции, требуется обучить модель находить нужный знак на изображении. Для каждого знака необходимо много данных, однако их классы распределены в датасете неравномерно. Соответствующая гистограмма распределения показана на рис. 1.

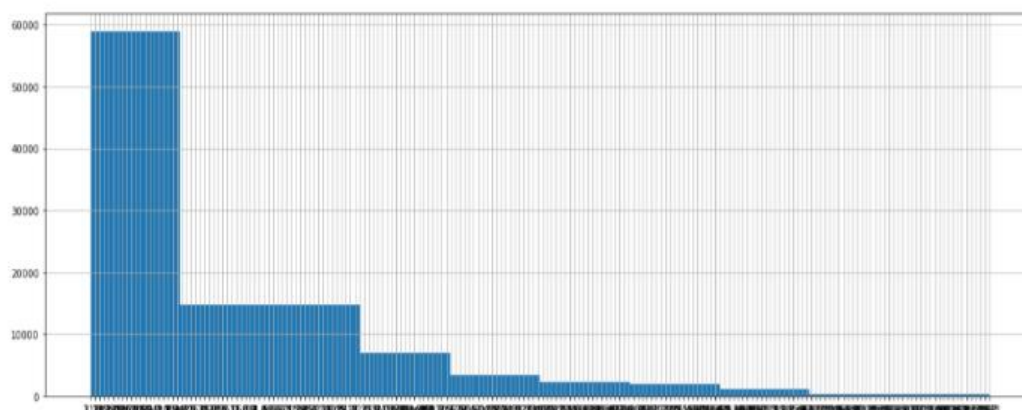


Рис. 1 Гистограмма распределения классов

Из этой гистограммы можно сделать вывод, что данных (фотографий) с редко встречающимися знаками недостаточно для обучения программы, поэтому число классов дорожных знаков придется сократить.

1.3 Подготовка данных

Как указано ранее, из рис. 1 можно сделать вывод, что среди данных RTSD имеется большое число типов дорожных знаков, которые имеют низкую частотность. Из-за этого модели будет сложно распознать эти редко встречающиеся знаки, потому удалим классы, которые встречаются реже 1500 раз. Гистограмма, соответствующая измененному датасету, приведена на рис. 2.

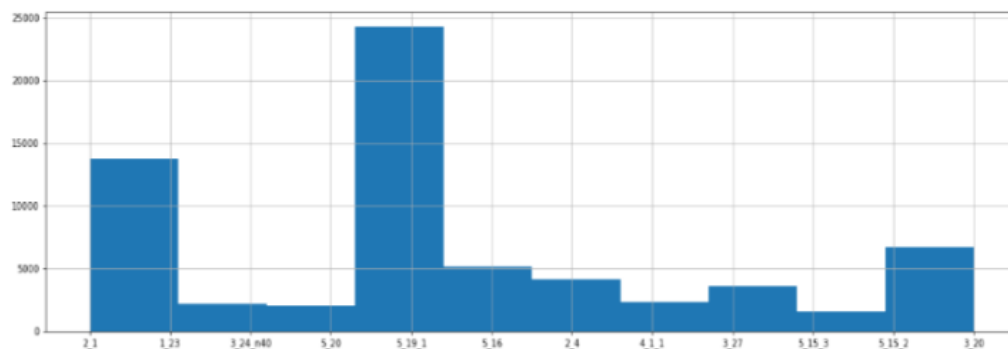


Рис. 2 Гистограмма распределения классов после удаления части данных

Таблица из файла full-gt.csv содержит семь колонок:

Таблица 1 Содержание измененного файла

Название	Содержание
filename	Содержит названия файла с фотоизображением
x_from	Координата по X левого верхнего угла прямоугольника. (угол знака на изображении)
y_from	координата по Y левого верхнего угла прямоугольника. (угол знака на изображении)
width	Ширина знака
height	Высота знака
sign_class	Тип знака
sign_id	Информация о знаке

2. Машинное обучение алгоритма

2.1 Работа с датасетом

Для расчета координат нижнего левого угла нужно из координаты по y верхнего угла вычесть высоту, координаты по x остается такой же. Для расчета координат верхнего правого угла нужно суммировать координату по x и ширину, координату по y оставляем без изменений (см. программу на рис. 3, здесь и далее программы на языке Python).

```
1 dataset['x1'], dataset['y1'] = dataset['x_from'], dataset['y_from'] - dataset['height']
2 dataset['x2'], dataset['y2'] = dataset['x_from'] + dataset['width'], dataset['y_from']
3 dataset = dataset.drop(['x_from', 'y_from', 'width', 'height'], axis=1)
```

Рис. 3 Расчет координат для SSD

Для обучения и тестирования модели необходимо разделить данные на обучающую и тестовую выборки. Возьмем 85% данных для обучения и 15% данных для тестирования, элементы выбираются случайным образом. Реализуется задача создания обучающего и тестового множеств с помощью программы, показанной на рис. 4.

```
1 from sklearn.model_selection import train_test_split
2 X_train, X_test, y_train, y_test = train_test_split(dataset.drop(['sign_class'], axis=1),
3                                                    dataset['sign_class'], test_size=0.15, random_state=42,
4                                                    stratify=dataset['sign_class'])
```

Рис. 4 Код, разделяющий множество

2.2 Обучение модели

Модель обучена в среде Google Colab, потому что обучение модели требует больших вычислительных мощностей и Google может предоставить такие мощности.

Перед началом обучения необходимо смонтировать диск для того, что программа могла иметь доступ к датасету на Google-диске и загрузить его рабочую память. Монтирует диск программа, приведенная на рис. 5.

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Mounted at /content/drive

Рис. 5 Монтирование диска

Далее необходимо загрузить предобученную модель из библиотеки Pytorch – современной библиотеки глубокого обучения. Данная библиотека стандартно использует несколько слоев для детекции ста классов (значит можно распознать до 100 знаков), но в рассматриваемой задаче мы ограничимся двенадцатью знаками.

В программе на рис. 6 указано тринадцать классов, двенадцать из них соответствуют детектируемым дорожным знакам, а еще один – это класс заднего фона.

```

1
2 num_classes = 13
3 # Step 1.
4 model = torchvision.models.detection.ssd300_vgg16(pretrained=False, num_classes=num_classes)
5
6 # Step 2, load the model state_dict and the default model's state_dict
7 mstate_dict = model.state_dict()
8 cstate_dict = torchvision.models.detection.ssd300_vgg16(pretrained=True).state_dict()
9
10 # Step 3.
11 for k in mstate_dict.keys():
12     if mstate_dict[k].shape != cstate_dict[k].shape:
13         cstate_dict.pop(k)
14 # Step 4.
15 model.load_state_dict(cstate_dict, strict=False)
16

```

Рис. 6 Подготовка модели к обучению

Первоначально датасет находится на Google-диске в заархивированном формате. У Google Colab имеется ограничение на количество запросов к Google-диску, потому обучать модель, просто указав путь до файла на нем, не получится. Для решения этой проблемы нужно скопировать архив в рабочую память проекта и разархивировать его, так как ограничений на количество обращений к рабочей памяти нет (см. программу на рис. 7).

```

1 # Прописываем путь к файлу с архивом
2 zip_file = '/content/drive/MyDrive/Diplom/yolov7_dataset.zip'
3
4 # Распаковываем архив
5 z = zipfile.ZipFile(zip_file, 'r')
6 z.extractall()

```

Рис. 7 Копирование и разархивирование датасета

В среде Google Colab модель обучалась в течении 60 эпох. Это означает, что программа распознавала знаки на всей выборке 60 раз.

На каждой эпохе для тестовой выборки считалась метрика Mean Average Precision (MAP) [4] – популярная метрика измерения точности детекторов объектов, показывающая среднюю точность веса нейрона в диапазоне от 0 до 1. На основе этой метрики для каждой эпохи выбиралась лучшая модель. Для минимизации функции ошибок использовался алгоритм Adaptive Moment Estimation (Adam). Adam – алгоритм адаптивной оптимизации, который создан специально для глубокого обучения нейронных сетей [5].. На рис. 8 графически представлено изменение функции ошибок в зависимости от эпохи.

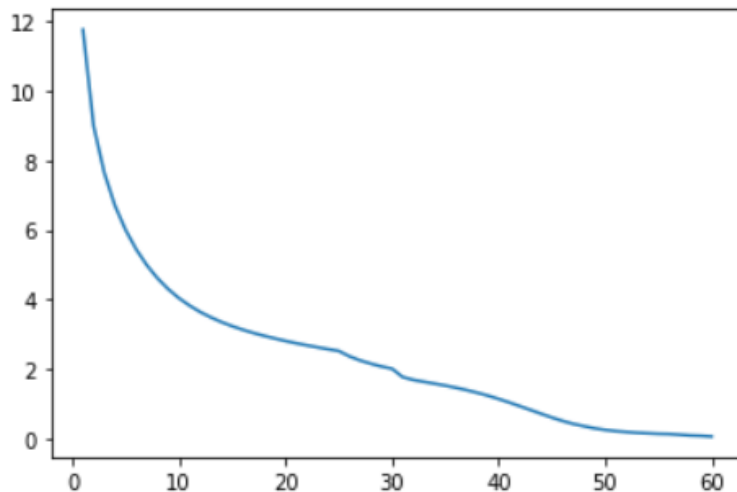


Рис. 8 Изменение функции ошибок в ходе обучения

2.3 Тестирование модели

Как можно видеть, в ходе обучения функция ошибки монотонно убывает, что может означать улучшение качества решения поставленной задачи. График на рис. 9 показывает изменение метрики MAP на тестовой выборке. Из него можно сделать вывод о преимущественном возрастании метрики MAP в ходе обучения, что может означать улучшение качества решения поставленной задачи. Максимальное значение метрики достигается на эпохе 55, а далее она убывает, что может говорить о переобучении.

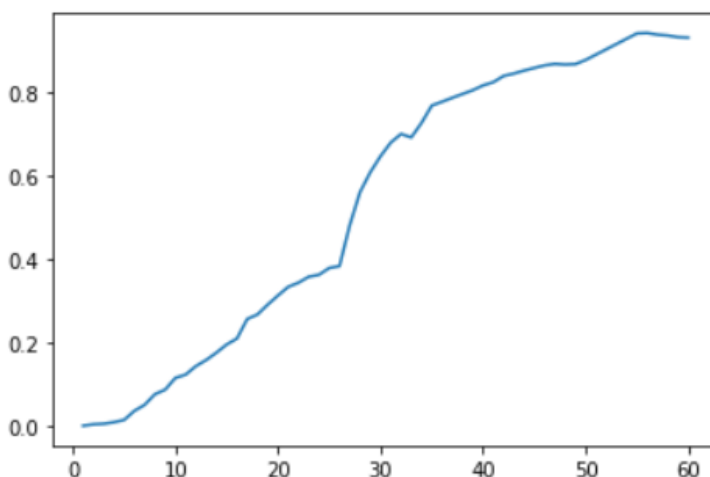


Рис. 9 Изменение метрики MAP на тесте в ходе обучения

После обучения сформированы матрицы ошибок для обучающей и тестовой выборок. Они представлены на рис. 10 и 11 соответственно. Из приведенных результатов делаем вывод, что используемая программа справилась с задачей.

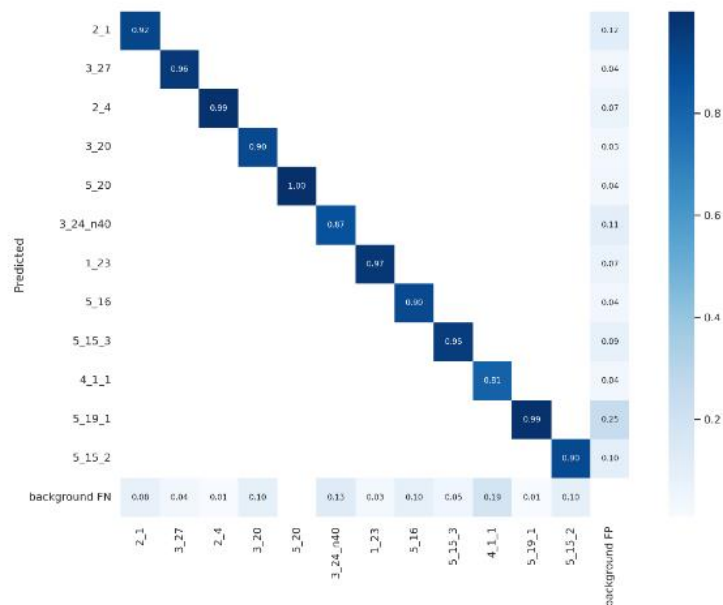


Рис. 10 Матрица обучающей выборки

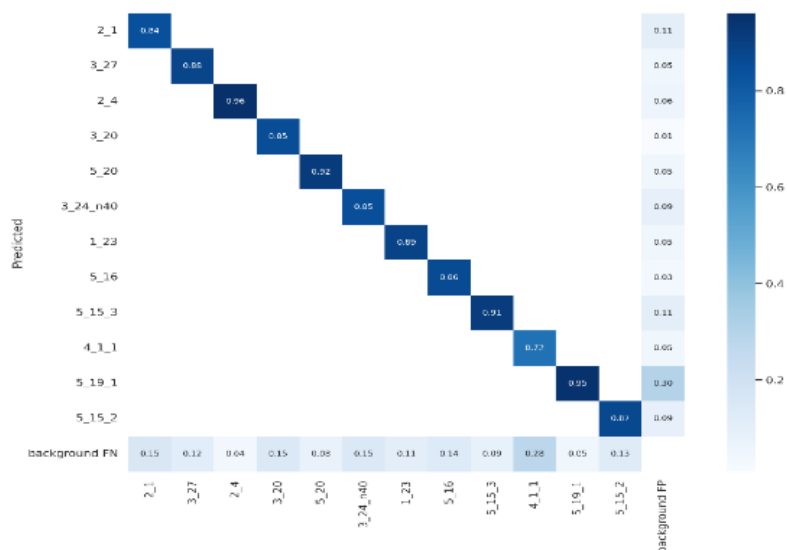


Рис. 11 Матрица тестовой выборки

Заключение

При выполнении работы была качественно решена задача детекции дорожных знаков. В качестве модели использовалась SSD. Перед обучением производились ряд операций над данными с целью подготовки датасета к обучению модели. Само обучение происходило в среде Google Colab. В итоге модель достаточно хорошо справилась с поставленной задачей, переобучения не наблюдалось, чаще всего модель путала знак с номером 5.19.1 с задним фоном.

Литература

- 1) Girshick R. Fast R-CNN // In Proc. IEEE International Conference on Computer Vision, Santiago, Chile, 2015, pp. 1440–1448. DOI: 10.1109/ICCV.2015.169.
- 2) Сазонтов И.Ю. Применение нейросетевой модели YOLOv7 в задаче детекции дорожных знаков // Гагаринские чтения – 2023 : Сборник тезисов докладов XLIX Международной молодежной научной конференции, Москва, 11–14 апреля 2023 года. – М.: Издательство «Перо», 2023. – С. 525–526.
- 3) full-gt.csv [Электронный ресурс].
URL: https://github.com/Rozen-Lab/Liu_et_al_Sup_Files
- 4) mAP (mean Average Precision) в детекции объектов [Электронный ресурс].
URL: <https://ml.i-neti.ru/map-mean-average-precision>
- 5) Adam Optimizer in Tensorflow [Электронный ресурс].
URL: <https://www.geeksforgeeks.org/adam-optimizer-in-tensorflow>

Информация об авторах

Сазонтов Иван Юрьевич – Магистрант 2-го года обучения кафедры математической кибернетики Московского авиационного института (национального исследовательского университета). isazontov@mail.ru 89936003081